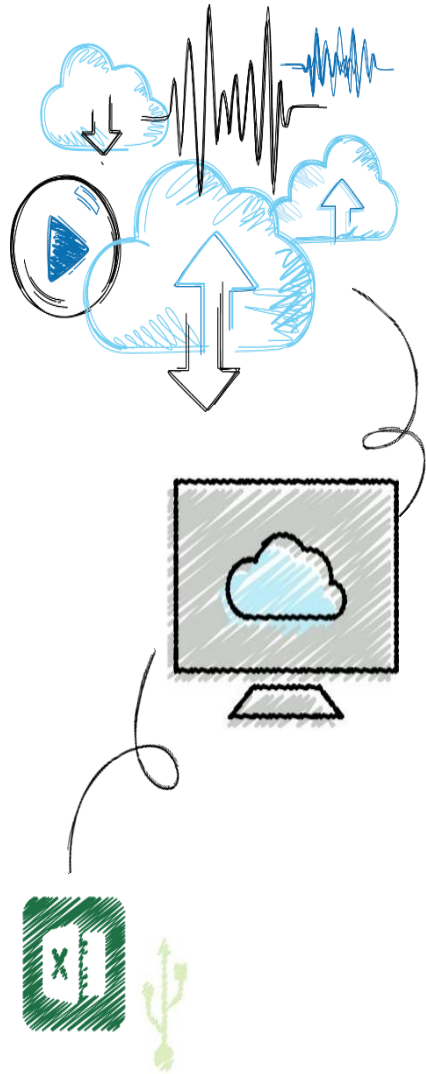


ICT703 Programming

Module 5

Text Files





Objectives

- Open a text file for output and write strings or numbers to the file
- Open a text file for input and read strings or numbers from the file
- Use library functions to access and navigate a file system



Text Files

- A text file is software object that stores data on permanent medium such as a disk (on your computer or in the cloud)
- When compared to keyboard input from human user, the main advantages of taking input data from a file are:
 - The data set can be much larger
 - The data can be input much more quickly and with less chance of error
 - The data can be used repeatedly with the same program or with different programs



Text Files and Their Format

- Using a text editor such as Notepad or Visual Studio Code, you can create, view, and save data in a text file
- A text file containing six floating-point numbers might look like:
34.6 22.33 66.75
77.12 21.44 99.01
- All data output to or input from a text file must be strings
 - Number must be converted to string before output



Writing Text to a File

- Data can be output to a text file using a **file** object
 - To **open** a file for output:
f = open("myfile.txt", 'w')
 - If file does not exist, it is created
 - If it already exists, Python opens it; when data are written to the file and the file is closed, any data previously existing in the file are erased
 - This statement writes a line of text to the file:
f.write("First line. \n")
 - When all outputs are finished, close the file:
f.close()
-



Writing Numbers to a File

- The **file** method **write** expects a string as an argument
 - Other types of data must first be converted to strings before being written to output file (e.g., using **str**)
- Code segment that illustrates the output of integers to a text file

```
import random
f = open("integers.txt", 'w')
for count in range(500):
    number = random.randint(1, 500)
    f.write(str(number) + '\n')
f.close()
```



Reading Text from a File (1 of 2)

- You open a file for input in a manner similar to opening a file for output

```
f=open("myfile.txt",'r')
```

If the path name is not accessible from the current working directory, Python raises an error

- There are several ways to read data from a file

- Example: the **read** method

```
text=f.read()
```

```
print(text)
```

First step is to open the file

Then you read the contents into a variable

Then you print that variable

If you try to print the filename, you will get the file details – not data



Reading Text from a File (2 of 3)

- After input is finished, **read** returns an empty string

```
f=open("myfile.txt",'r')
```

```
for line in f:  
    print(line)
```

- First step open the file
- Then use the for loop to loop through the lines



Reading Text from a File (3 of 3)

- Next code segment inputs lines of text with readline:

```
f=open("myfile.txt",'r')
line = f.readline()
while line != "":
    print(line)
    line = f.readline()
```

- First step open the file
 - Using a while loop, use the readline method to read each line.
 - Use a break once the end of file is reached
-



Reading Numbers from a File (1 of 3)

- In Python, string representations of integers and floating-point numbers can be converted to the numbers by using the functions **int** and **float**

```
f = open("integers.txt", 'r')
```

```
theSum = 0
```

```
for line in f:
```

```
    line = line.strip()
```

```
    number = int(line)
```

```
    theSum += number
```

```
print("The sum is", theSum)
```



Reading Numbers from a File (2 of 3)

- What happens if there are extra spaces in the file?
- You must extract the line details into a variable and loop through that.

```
f = open("integers.txt", 'r')
```

```
theSum = 0
```

```
for line in f:
```

```
    wordlist = line.split()
```

```
    for word in wordlist:
```

```
        number = int(word)
```

```
        theSum += number
```

```
print("The sum is", theSum)
```



Reading Methods

Method	What it Does
<code>open(filename, mode)</code>	Opens a file at the given filename and returns a file object. The mode can be 'r', 'w', 'rw', or 'a'. The last two values mean read/write and append.
<code>f.close()</code>	Closes an output file. Not needed for input files.
<code>f.write(aString)</code>	Outputs aString to a file.
<code>f.read()</code>	Inputs the contents of a file and returns them as a single string. Returns "" if the end of file is reached.
<code>f.readline()</code>	Inputs a line of text and returns it as a string, including the newline. Returns "" if the end of file is reached.



Accessing and Manipulating Files and Directories on Disk (1 of 5)

- The complete set of directories and files forms a tree-like structure
 - With a single **root directory** at the top and branches down to nested files and subdirectories
- You can access any other file or directory by using a **pathname**
 - When the chain starts with the root directory, it's called an **absolute pathname**
 - When chain starts from the current working directory, it's called a **relative pathname**

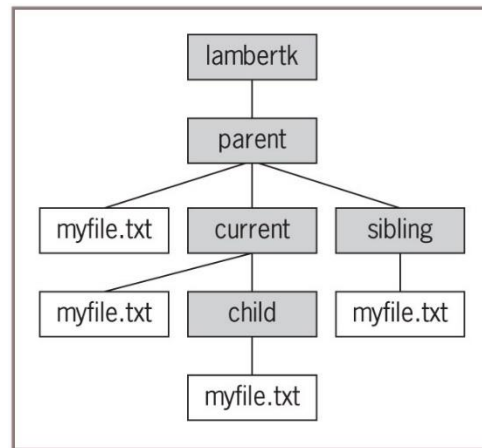


Figure 4-6 A portion of a file system



Accessing and Manipulating Files and Directories on Disk (2 of 5)

- To open files named myfile.txt in the child, parent, and sibling directories, where current is the current working directory, you could use relative pathnames:

childFile = open("child/myfile.txt", 'r')

parentFile = open("../myfile.txt", 'r')

siblingFile = open("../sibling/myfile.txt", 'r')

Pathname	Target Directory
myfile.txt	current
child/myfile.txt	child
../myfile.txt	parent
../sibling/myfile.txt	sibling



Accessing and Manipulating Files and Directories on Disk (3 of 5)

- When designing Python programs that interact with files, it's a good idea to include error recovery
- For example, before attempting to open a file for input, you should check to see if file exists
 - Function **os.path.exists** supports this checking
- Example: To print all of the names of files in the current working directory with a **.py** extension:

```
import os
currentDirectoryPath = os.getcwd()
listOfFileNames = os.listdir(currentDirectoryPath)
for name in listofFileNames:
    if ".py" in name:
        print(name)
```



Accessing and Manipulating Files and Directories on Disk (4 of 5)

os Module Function	What it Does
chdir(path)	Changes the current working directory to path .
getcwd()	Returns the path of the current working directory.
list dir(path)	Returns a list of the names in directory named path .
makedirs(path)	Creates a new directory named path and places it in the current working directory.
remove(path)	Removes the file named path from the current working directory.
rename(old, new)	Renames the file or directory named old to new .
rmdir(path)	Removes the directory named path from the current working directory.
sep	A variable that holds the separator character ('/' or '\') of the current file system.



Accessing and Manipulating Files and Directories on Disk (5 of 5)

os.path Module Function	What it Does
exists(path)	Returns True if path exists and False otherwise.
isdir(path)	Returns True if path names a directory and False otherwise.
isfile(path)	Returns True if path names a file and False otherwise.
getsize(path)	Returns the size of the object names by path in bytes.
normcase(path)	Converts path to a pathname appropriate for the current file system; for example, converts forward slashes to backslashes and letters to lowercase on a Windows system.